SignalsEverywhere Modding Guide

Adding and changing signals in Railroader

Tetz95 · October 2026 · SignalsEverywhere 1.4

Introduction

A guide to adding and changing signals in Railroader with SignalsEverywhere by Joo200.

SignalsEverywhere lets a mod add blocks, signals, control points and CTC panel levers, or change the ones the game ships with, all from JSON files. Its own docs list the properties. This guide explains how the pieces fit together, how patching works, and the traps that cost us hours, so you don't have to find them the hard way.

It's written for mod authors who are comfortable editing JSON. You don't need to know C#.

Contents

1. Getting started: what you need, the mod definition, and the edit–test loop.
2. How CTC works: blocks, signals, control points, intermediates and directions.
3. Locations and blocks: placing things on the track.
4. Signals: auto signals, predicate signals, heads and aspects.
5. Control points: interlockings, routes, intermediates and crossovers.
6. The CTC panel: drawing track, lights and levers.
7. Patching: changing what's already there, and the patch instructions.
8. RailForge: its extra checks and how to pass them.
9. Testing and debugging: tools, files and a symptom checklist.
10. Worked examples: a signal-only control point, a passing siding, and a junction added to an existing control point.

Quick reference: every property and patch instruction on one page.

Versions

Written against SignalsEverywhere 1.4 (October 2026). Some behavior described here is a bug with a fix waiting to be merged; those spots are marked, with a workaround. Tetz's SignalsEverywhere Fixes applies most of those fixes at run time, but write your mod so it works without it.

Credits

SignalsEverywhere is by Joo200. This guide is by Tetz95, from building the signal mods on Nexus (Branch Junctions, TRD Entry Signal, Alarka Branch Signals) and reading SignalsEverywhere's and the game's code along the way. Corrections are welcome as issues or pull requests.

License

This guide is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0); see LICENSE. You may share and adapt it, including in your own mods' docs, as long as you credit Tetz95 and link back here.

1. Getting started

What you need

- **Railroader** with the signal system available: in company mode the signal progressions must be bought; in sandbox, the CTC map feature must be on. SignalsEverywhere builds signals when the CTC panel exists, so nothing happens before that.
- **A mod loader that runs SignalsEverywhere**, one of:
- Railloader with StrangeCustoms;
- RailForge;
- FUSE. RailForge and FUSE each replace both Railloader and StrangeCustoms, so don't install those alongside them.
- **SignalsEverywhere**.
- A text editor that understands JSON. SignalsEverywhere ships JSON schemas (`signals.schema.json`, `ctc-panel-layout.schema.json`) in its mod folder. Point your file's `$schema` at them for hints; SignalsEverywhere removes the `$schema` key before it reads the file. Be aware that the schemas in SignalsEverywhere 1.4 reject some valid data, so treat schema errors as hints, not proof.

The mod definition

A signal mod is a data-only mod: a folder in `Railroader/Mods` with a `Definition.json` and your JSON files. Keep it data only: under FUSE, a folder that also has a Unity Mod Manager entry (an `info.json` with a DLL) doesn't get its `signals` and `ctcPanel` files passed to SignalsEverywhere, so its signals never appear. If your mod needs code, ship the code as a separate mod. SignalsEverywhere reads two mixinto types:

- `signals`: blocks, signals and control points.
- `ctcPanel`: the CTC panel layout.

```
{
  "manifestVersion": 5,
  "id": "yourname.my.signals",
  "name": "My Signals",
  "version": "1.0.0",
  "requires": [ { "id": "Joo.SignalsEverywhere" } ],
  "loadAfter": [ { "id": "Joo.SignalsEverywhere" } ],
  "mixintos": {
    "signals": [ "file(signals.json)" ],
    "ctcPanel": [ "file(ctc-panel.json)" ]
  }
}
```

- **`loadAfter`** controls the order patches are applied. List SignalsEverywhere and every mod whose signals you change or build on, so their changes are already there when yours are applied.
- A mixinto can be one string or a list. A list entry can also be an object with its own `requires`, for a file that only applies when another mod is installed (example below).

- You can split `signals` across several files. They are applied in the order listed. A second file is useful when one patch needs something an earlier one creates (see `$moveTo` in Patching).

A mixinto that only applies when another mod is installed:

```
"game-graph": [ { "mixinto": "file(signs.json)", "requires": [ "ALW.SceneryAssets" ] } ]
```

How SignalsEverywhere uses your files

When the map loads, SignalsEverywhere:

1. Reads every signal object the game already has and writes it out as one big JSON document. This is the **original data**.
2. Applies every mod's `signals` files to that document, in load order. The result is the **patched data**.
3. Rebuilds only the parts of the railroad a patch actually changed. It records which paths each patch touched; anything untouched is left exactly as the game built it.

The CTC panel works the same way, starting from SignalsEverywhere's own `CTCPanel-MainLine.json`.

This has two consequences you'll meet again and again:

- **Your file is a patch, not a full copy.** You only write what you add or change.
- **"Touched" matters.** A change that doesn't count as touching something doesn't reach the game, and touching some things has side effects. Patching covers both.

The edit–test loop

1. **Dump what's there.** Run `/signaldebug dump signals` in the game console (host only). SignalsEverywhere writes `signal-old.json` (original data) and `signal-patched.json` (after all mods) into its own mod folder. `/signaldebug dump panel` writes `panel-dump.json`. These are your map: real ids, real locations and the exact structure to patch.
2. **Write your patch.**
3. **Reload.** SignalsEverywhere's mod tab has **Reload Signal Definitions** and **Rebuild CTCPanel** buttons, so small changes don't need a restart. Restart the game when something looks stale.
4. **Check it.** Dump again and compare `signal-patched.json` with what you meant. Then check the game itself: patched data is not proof that a change reached the game (see Testing and debugging).
5. **Drive it.** Run a train through in both directions, in CTC and in ABS.

Finding track ids

Everything is placed by track **segment** ids (like `S4u5`) and switch **node** ids (like `Nuam`). The dumps give you every id the game's signals already use. For new places you need the track graph: get it from a track graph export, or from the `game-graph` file of a mod that changes that area.

Track ids mean nothing to a reader, so in your notes and commit messages always pair them with a plain name, like `Nuam (Cochran West switch)`.

2. How CTC works

Railroader's signal system is built from a few kinds of objects. SignalsEverywhere doesn't invent new ones; it lets you create and edit the game's own. Knowing what each one does makes the JSON make sense.

Features and modules

The signal data is grouped in two levels:

```
{
  "BK-AJ-HW": {             <- feature
    "CN-W": { ... },        <- module
    "CN-AL": { ... }
  }
}
```

- A **feature** is a stretch of railroad that the game switches on as a unit, like the signals between Bryson, Alarka Jct and Whittier (`BK-AJ-HW`). Put your modules in the feature that already covers that stretch. A new feature name creates a new group.
- A **module** is one place: a control point, or the stretch between two of them. Its key (`CN-W`) becomes the name of the game object that holds everything inside it.

A module can contain:

Key	What it holds
`blocks`	Track sections that detect trains.
`autoSignals`	Signals whose aspect comes from blocks and routes.
`predicateSignals`	Signals whose aspect comes from a list of conditions.
`interlocking`	A control point: switches, routes and exits. At most one per module.
`intermediate`	The automatic signals between two control points. At most one per module.
`crossover`	A control point that lets several routes be set at once. At most one per module.

Ids of blocks and signals must be unique across the whole railroad, not just the module.

Tying modules to a milestone

A feature is switched on by the game's own signal milestone for that stretch, and everything in it comes on together: modules you add to `BK-AJ-HW` appear as soon as the stock Bryson to Whittier signals do. To make your modules wait for a milestone of your own (a progression section in a `game-graph` file that unlocks a map feature), add a `map-feature-gates.json` to your mod folder:

```
{
  "gates": [
```

```
    {
      "mapFeature": "signals-alarka",
      "fallbackMapFeature": "alarka",
      "feature": "BK-AJ-HW",
      "modules": [ "AJ-BRANCH", "AB-CN", "CN-W", "CN-M", "CN-E", "CN-AL", "AL" ]
    }
  ]
}
```

- **`mapFeature`**: the map feature your milestone unlocks. The listed modules are on only while it's unlocked, and still need the stock milestone for their feature.
- **`fallbackMapFeature`** (optional): used instead when the running game's progression has no section that unlocks `mapFeature`, for example a start your progression file doesn't add the section to. Use the map feature that builds the track, so the signals still come on with the line.
- **`feature`** and **`modules`**: the feature your modules are in, and their keys.

SignalsEverywhere itself doesn't read this file. Tetz's SignalsEverywhere Fixes 1.8.0 or later reads it from every mod folder and switches the modules with the map feature, so the file works under RailForge, FUSE and Railloader alike. Without SignalsEverywhere Fixes the file does nothing and the modules come on with their feature, so make sure that's acceptable, or list SignalsEverywhere Fixes as a requirement. In sandbox, the modules follow the map feature as set in the sandbox options.

Blocks

A **block** is one or more stretches of track. When any car is on it, the block is **occupied**. Signals look at blocks to decide whether the way ahead is clear.

A block also shows occupied when a switch inside it is thrown by hand, or (in CTC) unlocked for hand operation. See Locations and blocks.

Signals

Every signal has a **direction**, `Left` or `Right`. This is the direction of travel it governs, using the same left and right as the CTC panel: a `Right` signal is for trains moving toward the right-hand side of the panel. On the main line the panel starts at Andrews on the left, so `Right` is eastbound. For a branch, it's whichever way you draw the branch on the panel.

There are two kinds:

- **Auto signals** (`autoSignals`) work out their aspect from the blocks they protect and, inside a control point, from the route that's lined.
- **Predicate signals** (`predicateSignals`) show a proceed aspect on a head when all of that head's conditions are true: a switch position, clear blocks, a direction set at a control point.

A signal has one, two or three **heads**. The top head shows the normal route, the second the diverging route, and the third a restricting move. The combination gives the aspect the game uses (and that the auto engineer obeys): Clear, Approach, Diverging Clear, Diverging Approach, Restricting or Stop. Signals explains how each kind decides.

Control points

An **interlocking** is a control point. It lists:

- **Switch sets**: the switches it controls, grouped so one panel knob throws a set together.
- **Outlets**: the ways out of the control point. Each has a direction, the block just beyond it, and the next signal beyond that block.
- **Routes**: for each combination of switch positions, which outlet a train leaves by in each direction.

In CTC, the dispatcher sets a route by turning the knobs on the panel lever and pressing the code button: the switches move, a direction is set, and the signals for that direction can clear. An interlocking with no switches at all is a **signal-only control point**, used to split a line or start a section.

Intermediates

Between two control points, the signals are automatic. An **intermediate** ties them together: the blocks in order from left to right, the signals in order from left to right, and the control point signal at each end. It lets a signal look ahead to the next one, and lets a train stop at an intermediate signal and then proceed (stop and proceed).

Directions and traffic

In CTC, when a route is set at a control point, the blocks on that route are given a **traffic direction**. An auto signal facing the other way shows Stop. This is what keeps two trains from being let onto the same single track from opposite ends. Blocks inside an intermediate don't hold a traffic direction themselves; the control points at each end decide.

CTC and ABS

The player can switch the railroad between **CTC** and **ABS**.

- In **CTC**, switches at control points are locked, routes come from the panel, and traffic directions apply.
- In **ABS**, there are no panel routes. Signals clear by themselves from the switch positions and block occupancy, and anyone can throw switches. Predicate conditions about a control point's direction count as true in ABS.

Test both. A signal that's right in CTC can be wrong in ABS, especially predicate signals.

The auto engineer

The auto engineer (AE) reads the same signals. It obeys Stop, slows for Approach and Restricting, and looks ahead for the next signal. That makes it a good tester: if the AE won't leave a siding or stops for no reason, look at the signal it's facing. Two game rules worth knowing:

- In Road mode the AE never throws switches; it stops before one that's set against it ("Switch Against").
- The AE's Approach speed and how long it holds it come from the game, not the signal mod.

- **Stop is absolute only at a control point.** A signal belongs to a control point when it sits in a module with an interlocking. Every other signal is an intermediate signal to the AE, and its Stop means stop and proceed: the AE stops about 15 m short, then passes it at 15 mph. A train already stopped within 40 m of it doesn't stop again, it just goes at 15 mph. At a control point signal showing Stop, the AE waits until it clears (or until the player tells it to pass). To make a lone signal absolute, see A signal on its own.

3. Locations and blocks

Locations

A location is a point on one track segment:

```
{ "segmentId": "S4u5", "distance": 72.242, "end": "End" }
```

- **`segmentId`**: the segment.
- **`end`**: `Start` or `End`, the end of the segment you measure from.
- **`distance`**: meters from that end.

The same point can be written from either end: on a 100 m segment, `20 from Start` is `80 from End`.

For signals, the end you measure from also sets which way the signal faces. The signal faces the end you measured from, so it's read by trains coming from that end. Measure a signal's location from the end trains approach it from.

A location must lie inside its segment. A distance longer than the segment is an error (RailForge refuses the whole module), and so is a segment id that doesn't exist. If another mod changes the track, segment lengths and ids can change under you. See RailForge.

Blocks

```
"blocks": {
  "cn-w": {
    "spans": [
      {
        "lower": { "segmentId": "Szr5", "distance": 21.031, "end": "Start" },
        "upper": { "segmentId": "S3xa", "distance": 4.707,  "end": "End" }
      },
      {
        "lower": { "segmentId": "Szr5", "distance": 21.031, "end": "Start" },
        "upper": { "segmentId": "Sls0", "distance": 4.595,  "end": "End" }
      }
    ],
    "thrownSwitchesSetOccupied": true
  }
}
```

Spans

Each **span** covers the track between two locations, `lower` and `upper`, following the track between them. A span can cross segments and switches. A block over a turnout needs one span per leg: the example above is the Cochran West switch, one span from the main line through the points to each leg.

If a block never shows occupied, a span's ends usually don't connect along the track. Check both ends, and try measuring one of them from the other end of its segment.

Placing block boundaries

Signals stand at block boundaries. A signal protects the block it faces into, so put the signal just before that block's boundary, on the approach side. We use 1 m:

```
       signal cn-we (Right) at Szr5 20.031 from Start
              v
--------------|--+=============== block cn-w ===========>
                 ^
       block boundary at Szr5 21.031 from Start
```

Blocks next to each other should meet at the same point, or overlap slightly. A gap leaves a car undetected.

Thrown and unlocked switches

`thrownSwitchesSetOccupied` (default `true`): when a switch inside the block is thrown by hand, the block shows occupied until the switch is set back. In CTC, a switch unlocked for hand operation also shows its block occupied.

- Keep it `true` for blocks on the main line or sidings, so a hand-thrown switch stops trains.
- When you patch a module's interlocking or crossover, SignalsEverywhere sets it to `false` on every block listed in that module, since those switches are thrown by the control point and are part of the route.

Inverse blocks

`createInverse: true` makes a second block named `<id>-inv` covering the same track, whose traffic direction is always the opposite of the first. It's for tracks that two control points see from opposite sides. Most mods don't need it.

Removing a block

Set it to `null` in a patch:

```
"blocks": { "old-block": null }
```

Anything that still names it (a signal, an outlet, an intermediate) needs changing too, or the game logs a warning and the reference is dropped.

Seeing blocks in game

SignalsEverywhere's mod tab has a **Blocks** dropdown. Pick a block and its spans are highlighted on the track. Use it to check every new block before you test with trains.

4. Signals

Properties every signal has

```
"cn-ws": {
  "direction": "Left",
  "headConfiguration": "Single",
  "location": { "segmentId": "Sls0", "distance": 3.595, "end": "End" },
  "leftSide": true,
  "offset": 3,
  "modelType": "Vanilla"
}
```

Property	Meaning
`direction`	`Left` or `Right`: the direction of travel it governs (see How CTC works).
`headConfiguration`	`Single`, `Double` or `Triple`. Auto signals can't be `Triple`; SignalsEverywhere changes it to `Double` with a warning.
`location`	Where it stands. Measure from the end trains approach from; that sets which way it faces. See Locations.
`leftSide`	`true` puts the mast on the left of the track, as seen by an approaching train. Default `false` (right).
`offset`	Meters from the track center, a whole number. Default 3. On the left side it's multiplied by 1.2.
`modelType`	Which signal model to copy. `Vanilla` (the default) uses the game's own. Another value is matched against the names of signals already in the world.

The key is the signal's id. An `id` property inside is ignored and overwritten by the key.

Signal direction, facing and side are separate, so check all three for every signal: `direction` decides which trains it governs, `location.end` decides which way it faces, and `leftSide` decides which side of the track it stands on.

How an aspect is built from heads

Each head shows green, yellow or red. The game combines them like this:

Top head	Second head	Third head	Aspect
green	any	any	Clear
yellow	any	any	Approach
red	green	any	Diverging Clear
red	yellow	any	Diverging Approach
red	red	green or yellow	Restricting
red	red	red	Stop

A head that may proceed shows green if the next signal on its route is showing anything but Stop, and yellow if the next signal shows Stop or there is no next signal. So **a missing next signal means Approach**, never Clear: leave `nextSignal` out only where the line really ends at a stop, like the end of signaled territory.

For an auto signal, a next signal showing a Diverging or Restricting aspect also gives yellow.

Auto signals

```
"autoSignals": {
  "cn-we": {
    "direction": "Right",
    "headConfiguration": "Double",
    "location": { "segmentId": "Szr5", "distance": 20.031, "end": "Start" },
    "blocks": [ "cn-w" ],
    "interlockingRouteMapping": [ 0, 1 ]
  }
}
```

- **`blocks`**: the blocks this signal protects, normally the block it faces into. If any of them is occupied, or (in CTC) has its traffic direction set against this signal, the signal shows Stop.
- **`interlockingRouteMapping`**: only for signals in a module with an interlocking. One route index per head: the top head follows route 0, the second head route 1. A head shows red unless its route is lined, then looks at that route's outlet in the signal's direction: the outlet's blocks must be clear, and its next signal decides green or yellow.

What the signal does depends on where it is:

Module has	Signal behaves as
An interlocking	A control point signal. Leave out `interlockingRouteMapping` and it never clears (the game logs an error).
An intermediate	An automatic signal. It looks ahead to the next signal in the intermediate in its direction, and shows Stop when the control point at the far end has a route set against it. Use `"interlockingRouteMapping": []`.
Neither	Shows Approach whenever its blocks are clear.

Extra mapping entries: SignalsEverywhere lets a `Double` auto signal list more than two routes. When the first two give Stop, it checks the extra ones and shows Diverging Approach if one of them is lined and clear. That's how a single diverging head can cover several diverging routes.

Predicate signals

A predicate signal gives each head a list of conditions. A head may proceed when all of its conditions are true; then its `nextCtcSignal` decides green (next signal not at Stop) or yellow.

The ids in this example are made up to show the shape.

```
"predicateSignals": {
  "el-we": {
    "direction": "Right",
    "headConfiguration": "Double",
    "location": { "segmentId": "S5xk", "distance": 12.0, "end": "Start" },
    "heads": [
      {
        "nextCtcSignal": "el-em",
        "predicates": [
          { "type": "Switch", "switchNode": "Nabc", "switchSetting": "Normal" },
          { "type": "Block", "blocks": [ "el-w", "el-mm" ] },
          { "type": "InterlockingTrafficDirection", "interlocking": "el-w", "direction": "Right" }
        ]
      },
      {
        "nextCtcSignal": "el-es",
        "predicates": [
          { "type": "Switch", "switchNode": "Nabc", "switchSetting": "Reversed" },
          { "type": "Block", "blocks": [ "el-w", "el-ms" ] },
          { "type": "InterlockingTrafficDirection", "interlocking": "el-w", "direction": "Right" }
        ]
      }
    ]
  }
}
```

Give it one entry in `heads` per head, top first.

Predicate types

These are the game's names. Some older docs use shorter ones (`Interlocking`, `Direction`); those don't work.

`type`	True when	Fields
`Switch`	The switch is in that position.	`switchNode`, `switchSetting` (`Normal` or `Reversed`)
`Block`	Every listed block is clear.	`blocks`
`InterlockingTrafficDirection`	In CTC: the control point's direction is the one given. Always true in ABS.	`interlocking`, `direction` (`Left`, `Right` or `None`)
`InterlockingTrafficDirectionIsNot`	In CTC: the control point's direction is not the one given, or (if you give `switchNode` and `switchSetting`) that switch isn't in that position. Always true in ABS.	`interlocking`, `direction`, optional `switchNode` + `switchSetting`
`AlwaysFalse`	Never. Holds a head at red.	none

Note that `InterlockingTrafficDirection` uses `None`, `Right` and `Left`, the traffic directions, not the signal directions.

When to use which kind

- **Auto signals** inside an interlocking are the normal choice for control point signals. The route table does the work, and traffic direction is handled for you.
- **Predicate signals** are for cases the route table can't express: a signal that depends on a switch outside the control point, or on another control point's direction. The game uses them at a few places (Bryson's west end).

A predicate signal that names a control point in another module has a load-order trap under RailForge. See RailForge.

Removing a signal

Set it to `null`:

```
"autoSignals": { "old-signal": null }
```

- **Auto signals**: safe.
- **Predicate signals**: SignalsEverywhere 1.4 crashes on a `null` predicate signal and skips the rest of that module (fixed by Tetz's SignalsEverywhere Fixes, pending upstream as PR #5). RailForge refuses the whole module instead. Rather than removing a predicate signal, rewrite it (or move it with `$moveTo`, but never a control point's signal; see RailForge). See Patching.

Signals are rebuilt, not edited

When a patch touches a signal at all, SignalsEverywhere deletes the game object and builds a new one. Anything that pointed at the old one keeps pointing at a deleted object unless it is rebuilt too. That includes interlocking outlets (`nextSignal`) and predicate heads (`nextCtcSignal`) in other modules. Such a stale pointer counts as "no next signal", so the signal behind it is stuck at Approach.

So whenever you change a signal, also touch everything that names it. Patching has the full rule.

5. Control points

Interlockings

```
"CN-W": {
  "interlocking": {
    "id": "cn-w",
    "displayName": "Cochran West",
    "switchSets": [ [ "Nuam" ] ],
    "outlets": [
      { "direction": "Left",  "blocks": [ "ab-cn" ], "nextSignal": "ajy-ee" },
      { "direction": "Right", "blocks": [ "cn-mm" ], "nextSignal": "cn-em" },
      { "direction": "Right", "blocks": [ "cn-ms" ], "nextSignal": "cn-es" }
    ],
    "routes": [
      { "switchFilters": [ "Normal" ],   "outletLeft": 0, "outletRight": 1 },
      { "switchFilters": [ "Reversed" ], "outletLeft": 0, "outletRight": 2 }
    ]
  },
  "blocks": { "cn-w": { ... } },
  "autoSignals": {
    "cn-we": { "direction": "Right", "headConfiguration": "Double", "blocks": [ "cn-w" ],
"interlockingRouteMapping": [ 0, 1 ], ... },
    "cn-wm": { "direction": "Left",  "headConfiguration": "Single", "blocks": [ "cn-w" ],
"interlockingRouteMapping": [ 0 ], ... },
    "cn-ws": { "direction": "Left",  "headConfiguration": "Single", "blocks": [ "cn-w" ],
"interlockingRouteMapping": [ 1 ], ... }
  }
}
```

This is the west end of a passing siding: one switch, the main line to the east (`cn-mm`), the siding to the east (`cn-ms`), and the single track to the west (`ab-cn`).

`id` and `displayName`

`id` is how signals, the panel and other mods refer to it. `displayName` is shown to the player. Interlocking ids must be unique railroad-wide.

`switchSets`

A list of sets; each set is a list of switch node ids. One set gets one knob on the panel, and every switch in a set is thrown together (both ends of a crossover, for example). The order matters: `switchFilters` in each route follows it.

A control point with no switches (`"switchSets": []`) is a signal-only control point.

`outlets`

The ways out. Each outlet has:

- `direction`: the direction of travel *leaving* the control point by this outlet.
- `blocks`: the block just past the control point on that side. These blocks get the traffic direction when a route is set, and must be clear for the signal to proceed. An empty list is allowed (dark territory beyond, for example).
- `nextSignal`: the next signal a train meets after leaving by this outlet. It decides green or yellow for the signal heads using this outlet. Use `null` only where there is no next signal; a head leading to `null` can never show better than Approach.

`routes`

Each route says: when the switches are in these positions, a train going `Left` leaves by outlet `outletLeft`, and a train going `Right` leaves by outlet `outletRight`.

- `switchFilters`: one entry per switch set, in the same order: `Normal`, `Reversed`, or `None` (doesn't matter).
- The first route whose filters match the current switch positions is the lined one.
- Routes are numbered from 0 in the order listed. Signals refer to them by number in `interlockingRouteMapping`.

Every switch position you want to be usable needs a route. Positions with no matching route can't be coded from the panel ("no route").

The interlocking's own blocks

Blocks in the same module as the interlocking are its **OS blocks** (the track over the switches). When a route is coded they all get the traffic direction, and the interlocking won't move a switch while any of them is occupied. Signals at the control point list them in `blocks`, so a train standing on the switches holds every signal at Stop.

Keep outlet blocks out of the control point's module. The game counts *every* block in the module as an OS block, whether or not it covers the switches. Once a route is coded, a train entering any of them cancels it, and while one is occupied the dispatcher can't code or cancel a route at that control point. So the blocks in the outlets (the track a train approaches on) belong in a module without an interlocking: the stretch's own module, or a neighbouring one, the way the base game keeps the blocks east of Bryson East in `GI-BR`. Under RailForge, prefer a module that comes before the control point's; forward references to blocks are accepted, but not to interlockings (see Forward references).

Signals at the control point

Each signal at the control point faces into the OS block and uses `interlockingRouteMapping` to say which route each head follows:

- Approaching the facing-point side (the side with the points), a `Double` signal with `[0, 1]`: top head for the normal route, second head for the diverging route.
- On each leg of the frog side, a `Single` signal mapped to the one route that leg belongs to: `[0]` for the main, `[1]` for the siding.

Coding a route

When the dispatcher codes a route, the game checks the OS blocks are clear and the switches can move, throws them, then sets the direction. Setting the direction gives traffic direction to the OS blocks and to the outlet's blocks, and fails if the next control point already has traffic set against it on that stretch. Once a train occupies an OS block, the direction is cleared again, and the dispatcher has to code the next move.

A signal on its own

The auto engineer treats Stop as absolute only at a signal that belongs to an interlocking; any other signal is stop and proceed (see The auto engineer). A predicate signal standing on its own, such as a block signal at a signal box, gets the AE's absolute stop when you put it in a module with a **switchless interlocking**: no switches, two empty outlets and one route. The interlocking needs no blocks of its own.

```
"COWEE-BOX": {
  "interlocking": {
    "id": "cowee-box",
    "displayName": "Cowee Signal Box",
    "switchSets": [],
    "outlets": [
      { "direction": "Left",  "blocks": [], "nextSignal": null },
      { "direction": "Right", "blocks": [], "nextSignal": null }
    ],
    "routes": [ { "switchFilters": [], "outletLeft": 0, "outletRight": 1 } ]
  },
  "predicateSignals": {
    "cowee-box-down": { "...": "a predicate signal with a Block predicate on cowee-down" }
  }
},
"COWEE-BLOCKS": {
  "blocks": { "cowee-down": { "...": "the block the signal protects" } }
}
```

The signal's aspect still comes from its own predicates, so it clears by itself when its block is free, in CTC and in ABS; nobody has to code it. The AE names it by the interlocking's `displayName` ("Stop, Cowee Signal Box"). Keep the block it protects in another module (here `COWEE-BLOCKS`): in the interlocking's module it would be an OS block. Don't use `InterlockingTrafficDirection` predicates on such a signal: no route is ever coded, so in CTC they never come true.

Intermediates

An intermediate covers the automatic signals between two control points.

```
"CN-AL": {
  "intermediate": {
    "id": "CN-AL",
    "blocks": [ "cn-al-1", "cn-al-2" ],
```

```
    "signals": [ "cn-al-wb", "cn-al-eb" ],
    "signalLeft": "cn-ee",
    "signalRight": "al-we"
  },
  "blocks": { "cn-al-1": { ... }, "cn-al-2": { ... } },
  "autoSignals": {
    "cn-al-wb": { "direction": "Left",  "blocks": [ "cn-al-1" ], "interlockingRouteMapping": [], ... },
    "cn-al-eb": { "direction": "Right", "blocks": [ "cn-al-2" ], "interlockingRouteMapping": [], ... }
  }
}
```

- **`id`**: use the module key.
- **`blocks`**: every block between the two control points, in order from left to right.
- **`signals`**: every automatic signal between them, in order from left to right by where they stand, both directions mixed.
- **`signalLeft`**: the first control point signal a `Left`-bound train meets past the left end. **`signalRight`**: the same for the right end.

An intermediate signal shows Stop when its block is occupied, and when the control point ahead of it has a route set against it. Otherwise it looks at the next signal in its direction: the next one in `signals`, or `signalLeft`/`signalRight` at the end.

Between two control points with no intermediate signals, you still need the blocks, but you don't need an intermediate. Put the blocks in the control points' outlets.

The control points at each end must name the intermediate's end blocks in their outlets, and their outlets' `nextSignal` should be the first intermediate signal in that direction. The links go both ways: if you add or move a signal on the line, update the intermediate and the control points.

The stock intermediates on a feature object. A few of the game's intermediates (between Whittier and Bryson) sit on the feature object instead of in a module, so they can't be patched. When you add a control point next to one, it keeps pointing at the old signal. Tetz's SignalsEverywhere Fixes relinks it (pending upstream as PR #8). Without it, avoid putting a new control point at the end of one of those intermediates.

Crossovers

A crossover is a control point that can have more than one route set at once, like a double crossover where each track can carry its own move. It has `switchSets`, `outlets` and `routes` like an interlocking, plus:

- **`routes[].usedBlocks`**: the blocks each route occupies. Two moves can be set at the same time only when their `usedBlocks` don't overlap.
- **`signalGroups`**: groups of signals, each with a `groupId`, its `signals`, the one `allowedDirection` it can be set to, and the `allowedRoutes` it can use. Each group gets its own direction knob on the panel.

Signals in a crossover are auto signals with `interlockingRouteMapping` (one route per head, like an interlocking). Predicate signals in a crossover can add `"crossoverGroup": "<groupId>"` to a predicate to tie it to a group's direction.

Crossovers are SignalsEverywhere's own addition, not part of the base game.

Make every `groupId` unique across the whole map, not just within the crossover. SignalsEverywhere saves a group's direction under `crossover:<groupId>:direction`, without the crossover's id. Two crossovers that both have a group `aeb` share one setting: clearing one clears the other's group too. Prefix the group with the crossover's id (`xo-aeb`).

One group per track and direction. A group is the signals on one track facing one way. Its `allowedRoutes` are the routes a train on that track, going that way, can take: for a double crossover between tracks A and B, the eastbound group on A gets "A straight" and "A over to B". Each group then needs its own `allowedDirection`.

Coding applies every knob. The code button reads all of the crossover's switch knobs and group knobs and codes them together. SignalsEverywhere first checks every group that's set; if any of them can't clear (its track is occupied, its route isn't lined, or it clashes with another group), nothing is coded, not even the groups that could. So before you code a new move, put any group whose train has already gone back to the middle.

Scissors crossovers. Where the two crossover legs cross each other in a diamond, a train on one leg fouls the other. Put both legs' blocks in each crossing route's `usedBlocks`, and the two diagonal moves can't be set together.

A double crossover

Tracks A and B, a crossover X1 from A (west) to B (east) and X2 from B (west) to A (east). Each crossover's switch set is the switch at both ends, thrown together. Outlets 0 and 1 are A and B on the west side, 2 and 3 on the east.

```
"crossover": {
  "id": "xo",
  "displayName": "Example Crossover",
  "switchSets": [
    [ "<X1 switch on A>", "<X1 switch on B>" ],
    [ "<X2 switch on B>", "<X2 switch on A>" ]
  ],
  "outlets": [
    { "direction": "Left",  "blocks": [ "xo-aw" ], "nextSignal": "<next signal west on A>" },
    { "direction": "Left",  "blocks": [ "xo-bw" ], "nextSignal": "<next signal west on B>" },
    { "direction": "Right", "blocks": [ "xo-ae" ], "nextSignal": "<next signal east on A>" },
    { "direction": "Right", "blocks": [ "xo-be" ], "nextSignal": "<next signal east on B>" }
  ],
  "routes": [
    { "switchFilters": [ "Normal", "Normal" ],   "outletLeft": 0, "outletRight": 2, "usedBlocks": [ "xo-a" ]
},
    { "switchFilters": [ "Normal", "Normal" ],   "outletLeft": 1, "outletRight": 3, "usedBlocks": [ "xo-b" ]
},
    { "switchFilters": [ "Reversed", "Normal" ], "outletLeft": 0, "outletRight": 3, "usedBlocks": [ "xo-a",
"xo-x1", "xo-b" ] },
    { "switchFilters": [ "Normal", "Reversed" ], "outletLeft": 1, "outletRight": 2, "usedBlocks": [ "xo-b",
"xo-x2", "xo-a" ] }
  ],
  "signalGroups": [
    { "groupId": "xo-aeb", "signals": [ "xo-aeb" ], "allowedDirection": "Right", "allowedRoutes": [ 0, 2 ] },
    { "groupId": "xo-beb", "signals": [ "xo-beb" ], "allowedDirection": "Right", "allowedRoutes": [ 1, 3 ] },
```

```
    { "groupId": "xo-awb", "signals": [ "xo-awb" ], "allowedDirection": "Left",  "allowedRoutes": [ 0, 3 ] },
    { "groupId": "xo-bwb", "signals": [ "xo-bwb" ], "allowedDirection": "Left",  "allowedRoutes": [ 1, 2 ] }
  ]
}
```

The plant blocks `xo-a`, `xo-b`, `xo-x1` and `xo-x2` go in the crossover's module; the outlet blocks don't (see The interlocking's own blocks). The signals `xo-aeb` (eastbound on A at the west end) and so on are auto signals in the same module, each head mapped to the routes its group can take. With both crossovers normal, eastbound on A and westbound on B can be cleared at the same time; "A over to B" uses both tracks' blocks, so it clears only on its own.

6. The CTC panel

The panel is drawn from a layout: a grid of track pieces, block lights and labels, with a lever (switch knobs, a direction knob and a code button) under each control point. SignalsEverywhere's own `CTCPanel-MainLine.json` draws the main line; mods patch it with a `ctcPanel` mixinto.

The layout

```
{
  "panel": {
    "Mainline": [ ... elements ... ],
    "Alarka Branch": [ ... elements ... ]
  }
}
```

Each key under `panel` is a **section**. `Mainline` is the game's main line. Any other key adds its own section, drawn below with a collapsible header. A branch with its own control points reads best as its own section.

Elements

```
{ "Type": "Light", "Block": "cn-w", "X": 4, "Y": 2, "Color": "red", "ShowTrack": false,
  "Interlock": { "Interlock": "cn-w", "SwitchLabels": [ "CW" ], "SignalLabel": "CW" } }
```

Field	Meaning
`Type`	What to draw (see below).
`X`, `Y`	Grid position. `Y` is the row; `X` the column (see How X works).
`Block`	For `Light`: the block whose occupancy it shows. For `Label`: text to show instead of `Id`.
`Id`	For `Label`: the text. For switch pieces: an id for highlighting.
`Color`	`white` (default) or `red`. Red lights mark OS blocks; red labels stand out.
`ShowTrack`	For `Light`: draw the track line behind the light. Default `true`.
`SwitchLabel`, `LabelOffsetX`, `LabelOffsetY`	A small blue label on a switch piece, nudged by the offsets.
`Interlock`	Puts a control point lever under this column (see below).
`Crossover`	Puts a crossover lever under this column.

Types

`Type`	Draws
`Label`	Text, usually a station name on row 0.

Type	Draws
`Light`	A block light, with track behind it unless `ShowTrack` is `false`.
`Track`	A plain horizontal track piece.
`TrackRise`, `TrackFall`	A diagonal piece going up or down to the next row.
`SwitchLeftTop`	Main track with a branch leaving to the upper right.
`SwitchRightTop`	Main track with a branch leaving to the upper left.
`SwitchLeftBottom`	Main track with a branch leaving to the lower left.
`SwitchRightBottom`	Main track with a branch leaving to the lower right.

Rows: the main line on the Mainline section sits on row 2, labels on row 0, sidings on rows 1 and 3. Follow the rows the surrounding panel uses.

Levers

An element with `Interlock` gets a lever in the control row under its column. Put it on the control point's OS block light.

```
"Interlock": {
  "Interlock": "cn-w",
  "SwitchLabels": [ "CW" ],
  "SignalLabel": "CW",
  "KnobOrder": [ 0 ]
}
```

- **`Interlock`**: the interlocking id.
- **`SwitchLabels`**: one label per switch knob, in `switchSets` order. One knob per switch set; a signal-only control point has none.
- **`SignalLabel`**: the label on the direction knob.
- **`KnobOrder`**: optional. Draws the switch knobs in a different order; must list every switch set.

Knob state is stored by id: `<interlocking>-0`, `<interlocking>-1`, … for switch knobs, `<interlocking>-D` for the direction knob. The game's own levers keep their old ids through `VanillaSwitchKnobIds` and `VanillaDirKnobId`; leave those as they are when you patch a stock lever.

For a crossover, `Crossover` takes `Crossover` (the id), `SwitchKnobOrder`, `SignalKnobOrder`, `SwitchLabels` and `SignalLabels` (one per signal group). Its knobs are stored as `<crossover>-0`, `<crossover>-1`, … for the switch sets and `<crossover>-<groupId>` for each group.

Drawing a crossover:

- Put the `Crossover` entry on a `Light` (an OS light with `"Color": "red"` and `"ShowTrack": false` works well), in the column where the crossover is drawn.
- A crossover lever is about four knobs wide (two switch knobs, four group knobs for a double crossover). Leave about ten columns between it and the next lever, with a light or track piece in every one of them, or the levers overlap. Columns count as distinct `X` values (see How X works): `X` 5 and `X` 16 with nothing

between are only one column apart.
- Draw each crossover with one switch piece. The piece goes on one track, and its diverging leg reaches the other; nothing is drawn on the other track in that column, so fill it with a `Track` or `Light` piece, or that track has a gap. For X1 from A (row 2) down to B (row 3): `SwitchRightBottom` at X 5 on row 2, and a `Track` at X 5 on row 3.
- For a scissors crossover, put both switch pieces in the same column (one `SwitchRightBottom` on the upper track, one `SwitchLeftTop` on the lower), so the legs cross in an X. Each fills the other's gap.

Levers only appear in CTC mode.

How X works

SignalsEverywhere doesn't use `X` as a position. It sorts every distinct `X` in a section and numbers them 1, 2, 3, …. So `X` only sets the **order** of columns, and you can slot a new column between 37 and 38 with `37.5` without moving anything.

That squashing has two consequences:

- Each distinct `X` becomes one grid cell, 36 px wide. Every element sharing an `X` lands in the same column.
- A lever is wider than a cell: about 70 px for one knob and 135 px for two. Levers in neighboring columns overlap unless there are enough columns between them. As a rule of thumb, two one-knob levers need 2 columns between their centers, a one-knob and a two-knob lever need 3, and two two-knob levers need 4. Add `Track` spacer pieces at new `X` values to make room.

All levers share one control row, whatever their `Y`, so two levers at the same `X` always collide.

Patching the panel

Sections are arrays, so patching uses the array instructions from Patching:

```
{
  "panel": {
    "Mainline": [
      { "$add": { "Type": "Light", "Block": "rg-mine", "X": 37.3, "Y": 3 } },
      { "$find": [ { "path": "Type", "value": "Light" }, { "path": "Block", "value": "aj-bk-1" } ],
        "X": 38 },
      { "$find": [ { "path": "Type", "value": "SwitchLeftTop" }, { "path": "X", "value": 49 } ],
        "$remove": true }
    ]
  }
}
```

- `$add` adds an element.
- `$find` matches existing elements by field values; the other fields in the object are merged into the match.
- `$remove: true` with `$find` deletes the match.

Match on fields every element in the section has, like `Type`, `X` and `Y`. `$find` throws an error on any element that lacks the field you test, and one error stops your panel file from applying (see Errors drop the whole file). Most elements have no `Id`, so `$find` on `Id` alone is risky.

For a new section, you can write plain elements. But if two mods add to the same new section, the second one must use `$add`, so use `$add` everywhere and it works whichever mod loads first.

Checking the panel

Use **Dump CTCPanel** in SignalsEverywhere's mod tab (or `/signaldebug dump panel`) to write the merged layout to `panel-dump.json` in SignalsEverywhere's folder. If the layout can't be read at all, SignalsEverywhere writes what it had to `ctcpanel-failed.json` and the panel won't open. **Rebuild CTCPanel** reloads the layout without restarting.

7. Patching

Your `signals` and `ctcPanel` files are patches. SignalsEverywhere merges them into the data it starts from (the game's signals, or the main line panel), one file at a time in load order. This page covers how the merge works, the instructions you can use, what each change rebuilds in the game, and the known problems.

The plain merge

With no instructions, a patch merges by key:

- **Objects** merge property by property. Properties you don't mention are kept. Keys are matched without regard to case.
- **Plain values** (strings, numbers, `true`/`false`) replace the old value.
- `null` as the value of a block, signal or module removes it (see the notes on removing).
- **Arrays can't be plain-merged.** To change an existing array you must say what to do: replace it, or edit its items with `$find`, `$index`, `$add` or `$append`. A plain array where one already exists is an error.

So this patch moves one existing signal and changes nothing else about it:

```
{
  "BR-W": {
    "BR-W": {
      "predicateSignals": {
        "br-we": {
          "location": { "segmentId": "Sabc", "distance": 12.5, "end": "Start" }
        }
      }
    }
  }
}
```

Instructions

Instructions are properties that start with `$`.

On an object

Instruction	Does
`"$replace": { ... }`	Replaces the whole object with the given value.
`"$moveTo": "<path>"`	Moves this object into another object, then merges the rest of the patch into it there.
`"$remove": true`	Removes the property. Works when the property holds an array or a plain value. On a property that holds an object it fails in 1.4 (see known problems).

`$replace` is the safe tool for anything structural, like rewriting an interlocking:

```
"interlocking": {
  "$replace": {
    "id": "aj-w",
    "displayName": "Alarka Jct West",
    "switchSets": [ [ "Nowf" ] ],
    "outlets": [ ... ],
    "routes": [ ... ]
  }
}
```

`$moveTo` takes a JSON path from the root of the document, quoted in brackets because ids contain dashes and spaces:

```
{ "MY-FEATURE": { "OLD-MODULE": { "autoSignals": {
  "my-signal": { "$moveTo": "['MY-FEATURE']['NEW-MODULE']['autoSignals']" }
} } } }
```

The destination must already exist when this patch is applied. If your own mod creates it, create it in an earlier file and move in a later one. Moving a signal deletes the old game object and builds it again in the new module.

Never move a control point's signal out of that control point's module. A signal belongs to the interlocking of the module it's in. Moved anywhere else, the game treats it as an intermediate signal, and the auto engineer will pass it at Stop.

On an array

Each item in the patch array is an object of instructions.

Item	Does
`{ "$add": <value> }`	Appends one item.
`{ "$append": [ <values> ] }`	Appends several.
`{ "$find": [ <conditions> ], ...fields }`	Finds the first matching item and merges the fields into it.
`{ "$index": 2, ...fields }`	The same, for the item at that position (from 0).
`{ "$find": [...], "$selectAll": true, ...fields }`	Every matching item, not just the first.
`{ "$find": [...], "$replace": <value> }`	Replaces the matching item.
`{ "$find": [...], "$remove": true }`	Removes the matching item.
`{ "$find": [...], "$clone": true, ...fields }`	Copies the matching item, merges the fields into the copy, and appends it.
`"$optional": true`	With `$find` or `$index`: if nothing matches, skip quietly instead of failing.

To replace an existing array outright, use `$replace` on the property, not an array: `"blocks": { "$replace": [ "a", "b" ] }`.

`$find` conditions

```
"$find": [
  { "path": "direction", "value": "Left" },
  { "path": "nextSignal", "comparison": "StartsWith", "value": "aj-" }
]
```

- **`path`**: a JSON path inside the item (`"direction"`, `"blocks[0]"`).
- **`comparison`**: `Equals` (default), `NotEquals`, `StartsWith`, `EndsWith` or `Contains`. Text comparisons ignore case.
- **`value`**: what to compare against.

All conditions must match. **Every item in the array must have every path you test**: an item without it throws an error instead of simply not matching, and the error drops your whole file.

Errors drop the whole file

If any instruction in a file fails (a `$find` with no match, an unsupported instruction, a bad path), SignalsEverywhere logs the error and stops applying **that file**. Everything after the error is lost, and you can't count on what came before it either. A mod whose changes all silently vanish usually has one broken patch. Keep risky or optional patches in a separate file, and use `$optional` where something may legitimately be missing.

What gets rebuilt

SignalsEverywhere records every path a patch changes, then rebuilds only what was touched:

What you touch	What happens in the game
Nothing in a module	The module is left exactly as it was.
A block	Its spans are replaced in place. Things that point at the block keep working.
A signal	The signal is **deleted and built again**.
A module's `interlocking` / `crossover`	The same object is kept and its switch sets, outlets and routes are rewritten from the patched data.
A module's `intermediate`	Rebuilt the same way.
Anything else in a module, but not its `interlocking`, `intermediate` or `crossover`	See the next rule.

Two rules follow from this.

Touch the control point when you patch its module. In SignalsEverywhere 1.4, when a module is rebuilt and its interlocking, intermediate or crossover was not touched, that component is **deleted**. RailForge prevents this, but under plain Railloader it happens. So whenever you patch anything in a module that has one, also touch it. The easiest harmless touch is replacing an array with its current value:

```
"interlocking": { "switchSets": { "$replace": [ [ "Nuam" ] ] } }
```

Touch everything that names a signal you changed. A rebuilt signal is a new object. Interlocking outlets (`nextSignal`) and predicate heads (`nextCtcSignal`) that aren't rebuilt still point at the deleted one, which counts as "no next signal", so the signal behind it is stuck at Approach. Intermediates are relinked by id on every build, so they're fine. So when you change a signal, find every outlet and predicate head that names it (search the dump for its id) and touch those control points or signals too, even if the value is the same.

Removing things

To remove	Write	Notes
A block	`"block-id": null`	Update everything that names it.
An auto signal	`"signal-id": null`	Safe.
A predicate signal	`"signal-id": null`	Crashes SignalsEverywhere 1.4 (see below). Rewrite it instead, or `$moveTo` it if it isn't a control point signal.
A whole module	`"MODULE": null`	Deletes the module's game object and everything in it.
An array item	`{ "$find": [...], "$remove": true }`	Works.
A property	`"prop": { "$remove": true }`	Fine for an array or plain value. For a property holding an object it fails in 1.4 (see below); `$replace` the parent instead.

Map mods that change stock track

A map mod that removes or re-lays stock track (new segment ids for the main line, say) leaves the stock signals there pointing at segments that no longer exist. The game doesn't drop them; their blocks lose their spans (FUSE logs "removed TrackSpan … upper/lower endpoint segment is missing", with the block's name), and signals added to the same group alongside them misbehave in ways that look random. Signals that worked before stop working once the track under the stock ones goes.

Before signalling the new track:

1. List the stock blocks whose spans use a removed segment (compare `signal-old.json` against the map mod's removals).
2. `null` every stock module that holds them. A whole module goes at once, with its intermediate and interlocking.
3. Check the stock modules you keep for anything that names what you removed: an outlet's `blocks` or `nextSignal`, an intermediate's ends. `$replace` those outlets, typically with `"blocks": []` and `"nextSignal": null` where the line now ends.
4. Then add your own modules on the new track, in a patch that loads after the removals.

Known problems

These are bugs in SignalsEverywhere 1.4 with fixes sent upstream. Tetz's SignalsEverywhere Fixes applies the first four at run time. Write your mod so it works without it.

Problem	Symptom	Workaround	Upstream
Array item edits don't count as a touch. `$find`, `$index`, `$add` and `$append` inside an existing array change the patched data but are recorded under the wrong path, so the component isn't rebuilt.	Your edit shows in `signal-patched.json`, but the game behaves as before.	Also put a harmless `$replace` on the same component, such as `"switchSets": { "$replace": [ ...same value... ] }`.	PR #7
Only one interlocking per module is seen. A stock module with two (Alarka Jct: `aj-e` and `aj-w`) only exposes the first.	You can't patch `aj-w`; patching `aj-e` adds a second interlocking.	Needs Tetz's SignalsEverywhere Fixes (moves the second one into its own module, `AJ-W`).	PR #4
`null` predicate signal crashes.	The predicate signal is deleted, then the rest of the module is skipped.	Rewrite it instead of removing it.	PR #5
Stock intermediates on a feature object keep pointing at old signals next to a new control point.	Errors about aspects; the new control point's signals clear without a route.	See Intermediates.	PR #8
`$remove` on a property that holds an object throws "Unsupported patch instructions".	The file stops applying at that point.	`$replace` the parent object, or `$remove` the whole array item.	PR #9

8. RailForge

RailForge replaces Railloader and StrangeCustoms: it loads Railloader-style mods, including SignalsEverywhere, and adds its own handling for signal mods. Most players use it, so test under it. It changes three things.

It keeps untouched control points

Under RailForge, patching part of a module no longer deletes an interlocking, intermediate or crossover you didn't touch (see What gets rebuilt). Keep touching them anyway, so your mod works under plain Railloader too.

It checks every patched module first

Before SignalsEverywhere builds anything, RailForge checks each module a mod patches. If a module fails a check, RailForge **defers** it: it removes your changes to that module, leaves the game's version running, and builds nothing new there. Other modules still apply. Nothing about this shows on screen, so a deferred module looks exactly like a patch that didn't work.

RailForge writes the reason to its support report, `Mods/RTM.RailForge/RailForge-support-report.txt`, under `## Signal authoring`. The section is rewritten every time a map loads and lists each deferred module with the path and the message. **When a module's changes are missing under RailForge, read that section first.**

The checks we've run into:

Forward references

Everything a signal or predicate names must already exist **when that signal is created**: in the same module, in an earlier module, or in the game's own data. Modules are created in order, feature by feature, with the game's modules first and new modules after them in the order they appear in the patched data.

This bites predicate signals that name another control point. If module `AB-S` has a predicate signal naming interlocking `ab-w`, and `ab-w` is in module `AB-W` which comes later, RailForge defers `AB-S`. Some RailForge versions also apply it to the game's own control points: a stock predicate signal in `BR-E` that names `br-w` in `BR-W` (which comes after it) gets deferred as soon as your mod patches it. SignalsEverywhere itself has no trouble with a control point that's already in the game, because it indexes every existing interlocking before building anything; the check is stricter than it needs to be. RailForge 0.14.71 no longer flags this case.

Fixes:

- **For a new control point:** define it in a module that comes before every signal that names it. A new module is added after the existing ones, in the order they appear.
- **For a signal at a control point:** keep it in its control point's module. **Don't move it to another module to get past this check.** A signal only belongs to a control point when it sits in that control point's module; anywhere else, the game treats it as an intermediate signal. The auto engineer then stops at it and passes it at Stop, as it would at any intermediate signal. Tetz's SignalsEverywhere Fixes 1.4.0 lets RailForge accept a reference to a control point that's already in the game, which covers the stock case above.

- **For a signal that isn't part of a control point** (an intermediate, or a stand-alone predicate signal), moving it to a later module with `$moveTo` is fine. The destination must already exist, so create it in one `signals` file and move into it from a second file.

Forward references to **blocks** are fine.

Locations outside their segment

Every location you patch must lie within its segment as the track exists in the game right now. Other mods can change track: a mod that shortens a segment can make a stock signal's location invalid the moment you patch that signal, even if you didn't move it. If a module is deferred for a location you never changed, check whether another track mod moved or shortened that segment, and write the location from the other end or on the new segment.

Removing predicate signals

RailForge refuses any patch that sets a predicate signal to `null`, because of the SignalsEverywhere crash (see Known problems), and defers the module. Tetz's SignalsEverywhere Fixes lets that one check through once the crash is fixed. Without it, don't remove predicate signals: move or rewrite them.

Load order

RailForge orders mods by `loadAfter`. List every mod whose signals or panel you patch, and every mod that changes track your signals stand on, so your patches see their changes.

9. Testing and debugging

Tools

SignalsEverywhere's mod tab (in the game's mod settings):

Button	Does
Reload Signal Definitions	Re-reads every `signals` file and rebuilds.
Dump Signals	Writes `signal-old.json` and `signal-patched.json` to SignalsEverywhere's folder.
Rebuild CTCPanel	Re-reads every `ctcPanel` file and redraws the panel.
Dump CTCPanel	Writes `panel-dump.json`.
Blocks (dropdown)	Highlights the chosen block's track on the map.

Console commands (host only):

- `/signaldebug dump signals`: same as Dump Signals.
- `/signaldebug dump panel`: same as Dump CTCPanel.

Files in SignalsEverywhere's mod folder:

File	Holds
`signal-old.json`	The game's signals before any mod. Your source of real ids and locations.
`signal-patched.json`	After every mod's patches. Shows what SignalsEverywhere **meant** to build.
`panel-dump.json`	The merged panel layout.
`ctcpanel-failed.json`	Written when the panel layout can't be read at all.

Logs: SignalsEverywhere logs patch errors ("Error loading mixinto ..."), missing ids ("Block with ID 'x' not found in patching context") and what it created. The missing-id warning isn't always an error: a new predicate signal looks up its blocks once while it's being built, before the blocks defined beside it in the same patch exist, and logs this warning for each of them. A second pass finds them. If the block is in `signal-patched.json`, the warning is harmless; if it isn't, the id is wrong or the block was removed. Under RailForge, also read the `## Signal authoring` section of `Mods/RTM.RailForge/RailForge-support-report.txt` (see RailForge).

Patched data is not proof

`signal-patched.json` shows the result of the merge. Whether that reached the game depends on what counted as touched (see What gets rebuilt) and on RailForge's checks. A change can be in the patched data and still not be in the game. Always confirm on the railroad: the panel, the signals, and a train.

A test run

For every control point you add or change:

1. **Blocks**: highlight each new block in the mod tab. Then run a train through: each block light on the panel should come on as the train enters and go off when the last car leaves.
2. **Each route, each direction (CTC)**: code the route, check the right signal clears and the others stay at Stop. Check the aspect: Clear or Approach on the normal route, Diverging on the diverging one.
3. **Next signal**: with the next signal at Stop, a signal should show Approach. With it clear, Clear. A signal that never shows Clear has a missing or stale next signal.
4. **Opposing moves**: set a route one way, then try to code the opposite direction at the next control point. It should refuse.
5. **Occupancy**: put a car on the OS block. Every signal at that control point should go to Stop, and the switches shouldn't move.
6. **Approach**: code a route, then put a car in the block a train approaches on (an outlet block). The signal should stay clear, and the dispatcher should still be able to cancel and recode the route. If the route drops, that block is in the control point's module.
7. **Hand-thrown switch**: unlock a switch in CTC or throw one in ABS. Its block should show occupied.
8. **ABS**: switch the railroad to ABS and repeat the basic moves. Predicate signals are where ABS surprises happen.
9. **Auto engineer**: send an AE train through on Road mode both ways. It should obey every aspect without stopping where it shouldn't.
10. **Reload the save** and check the routes and directions came back.

Symptoms and causes

Symptom	Likely cause
None of the mod's changes appear.	One broken instruction stopped the whole file. Check the log for "Error loading mixinto".
One module's changes are missing under RailForge.	RailForge deferred it. Read `## Signal authoring` in the support report.
The change is in `signal-patched.json` but not in the game.	An array item edit (`$find`/`$index`/`$add`) that didn't count as a touch. Add a harmless `$replace` to the same component.
A signal is stuck at Approach and never shows Clear.	Its next signal is `null`, or points at a signal that was rebuilt (a stale reference). Touch the control point or predicate signal that names it.
A route drops as soon as a train enters the block approaching the control point, and the lever won't code or cancel while it's there.	That outlet block is defined in the control point's module, which makes it an OS block. Move it to a module without an interlocking.
The AE stops at a red signal, then passes it at 15 mph (or passes at once if it was stopped close to it).	Stop and proceed: the signal isn't in a module with an interlocking, so the AE treats it as an intermediate signal. That's right for block signals; for an absolute signal, see A signal on its own.
Signals near a map mod's new or moved track never work, or stock signals there show odd aspects.	The map mod removed or re-laid track the stock signals stand on. See Map mods that change stock track.
A control point's lever does nothing ("no route").	No route matches the current switch positions, or `switchFilters` don't line up with `switchSets`.
A control point vanished after a patch (plain Railloader).	You patched its module without touching its interlocking/intermediate.
A block never shows occupied.	A span's ends don't connect along the track. Try measuring one end from the other end of its segment.

Symptom	Likely cause
A block shows occupied with no train.	It overlaps a hand-thrown switch, or another train's block. Check with the Blocks highlight.
A signal faces the wrong way.	Its location is measured from the wrong end.
A signal stands on the wrong side or in the track.	`leftSide` or `offset`.
A signal clears for the wrong direction.	Its `direction`, or the outlet direction in the interlocking.
A control point signal never clears, with an error about an empty `interlockingRouteMapping`.	The auto signal is missing its route mapping.
Intermediate signals stay red after the train has passed, or clear against a route.	The intermediate's block or signal order isn't left to right, or its `signalLeft`/`signalRight` don't match the control points.
The panel won't open.	The panel layout couldn't be read; see `ctcpanel-failed.json` and the log.
Levers overlap on the panel.	Columns too close together. See How X works.

10. Worked examples

These come from real mods (Tetz's Alarka Branch Signals and Branch Junctions). Locations are trimmed to keep the focus on the structure; the full files are in those mods.

Example 1: a signal-only control point

Alarka Jct is yard limits, and the Alarka Branch leaves from its wye. A signal-only control point (`ajy`) marks where signaled territory starts: trains leaving the yard get a signal onto the branch, and trains coming off the branch get a signal into the yard.

```
{
  "BK-AJ-HW": {
    "AJ-BRANCH": {
      "$replace": {
        "interlocking": {
          "id": "ajy",
          "displayName": "Alarka Jct Yard Limit",
          "switchSets": [],
          "outlets": [
            { "direction": "Left",  "blocks": [],         "nextSignal": null },
            { "direction": "Right", "blocks": [ "ab-cn" ], "nextSignal": "cn-we" }
          ],
          "routes": [
            { "switchFilters": [], "outletLeft": 0, "outletRight": 1 }
          ]
        },
        "blocks": {
          "ajy": { "spans": [ { "lower": { ... }, "upper": { ... } } ] }
        },
        "autoSignals": {
          "ajy-ee": { "direction": "Left",  "headConfiguration": "Single", "blocks": [ "ajy" ],
"interlockingRouteMapping": [ 0 ], "location": { ... } },
          "ajy-wm": { "direction": "Right", "headConfiguration": "Single", "blocks": [ "ajy" ],
"interlockingRouteMapping": [ 0 ], "location": { ... } }
        },
        "predicateSignals": {}
      }
    }
  }
}
```

Points to notice:

- **No switches, one route.** `switchSets` and `switchFilters` are empty, so route 0 is always lined.

- **The yard side has no blocks and no next signal.** Westbound (`Left`) trains go into yard limits, where there's nothing to detect. `ajy-ee` can only show Approach, which is right: the train is entering yard limits.
- **The OS block `ajy`** is a short block between the two signals. Both signals protect it, so a train standing there holds both at Stop.
- **The whole module is `$replace`d.** This mod replaced a stock module with a different design, so it rewrites it from scratch instead of patching piece by piece. `$replace` touches everything, which also avoids the touch problems from Patching.
- **`ab-cn` is shared.** It's the single track between `ajy` and Cochran West. It's listed in `ajy`'s right outlet and in `cn-w`'s left outlet, and lives in its own module (`AB-CN`) with no intermediate, because there are no signals between the two control points.

Example 2: a passing siding

Cochran has a passing siding with a control point at each end and nothing else. Four modules:

Module	Holds
`CN-W`	Interlocking `cn-w` (switch `Nuam`), OS block `cn-w`, signals `cn-we`, `cn-wm`, `cn-ws`.
`CN-M`	Blocks `cn-mm` (main) and `cn-ms` (siding). No control point.
`CN-E`	Interlocking `cn-e` (switch `Natz`), OS block `cn-e`, signals `cn-ee`, `cn-em`, `cn-es`.
`CN-AL`	The intermediate to Alarka.

The two interlockings mirror each other:

```
"cn-w": {
  "switchSets": [ [ "Nuam" ] ],
  "outlets": [
    { "direction": "Left",  "blocks": [ "ab-cn" ], "nextSignal": "ajy-ee" },
    { "direction": "Right", "blocks": [ "cn-mm" ], "nextSignal": "cn-em" },
    { "direction": "Right", "blocks": [ "cn-ms" ], "nextSignal": "cn-es" }
  ],
  "routes": [
    { "switchFilters": [ "Normal" ],   "outletLeft": 0, "outletRight": 1 },
    { "switchFilters": [ "Reversed" ], "outletLeft": 0, "outletRight": 2 }
  ]
},
"cn-e": {
  "switchSets": [ [ "Natz" ] ],
  "outlets": [
    { "direction": "Right", "blocks": [ "cn-al-1" ], "nextSignal": "cn-al-eb" },
    { "direction": "Left",  "blocks": [ "cn-mm" ],   "nextSignal": "cn-wm" },
    { "direction": "Left",  "blocks": [ "cn-ms" ],   "nextSignal": "cn-ws" }
  ],
  "routes": [
    { "switchFilters": [ "Normal" ],   "outletLeft": 1, "outletRight": 0 },
    { "switchFilters": [ "Reversed" ], "outletLeft": 2, "outletRight": 0 }
```

```
  ]
}
```

And the signals at `cn-w`:

Signal	Direction	Heads	Mapping	Stands
`cn-we`	Right	Double	`[0, 1]`	On the single track, facing the points. Top head for the main, second for the siding.
`cn-wm`	Left	Single	`[0]`	On the main, facing the frog.
`cn-ws`	Left	Single	`[1]`	On the siding, facing the frog, on the left (`"leftSide": true`) because the siding is on that side.

Points to notice:

- **Each outlet's next signal is the signal at the far end of that track.** A train leaving `cn-w` eastbound on the siding meets `cn-es` next, so outlet 2's next signal is `cn-es`, and `cn-we`'s second head looks at it.
- **The middle blocks belong to both control points.** `cn-mm` and `cn-ms` are outlets of both, which is how a route set at one end blocks an opposing route from the other.
- **Route numbers are per interlocking.** Route 1 at `cn-w` is the siding; at `cn-e` it's also the siding, but only because the routes were listed the same way.

Its panel section

```
{ "panel": { "Alarka Branch": [
  { "$add": { "Type": "Light", "Block": "cn-w", "X": 4, "Y": 2, "Color": "red", "ShowTrack": false,
              "Interlock": { "Interlock": "cn-w", "SwitchLabels": [ "CW" ], "SignalLabel": "CW" } } },
  { "$add": { "Type": "SwitchRightBottom", "X": 4, "Y": 2, "Id": "CW" } },
  { "$add": { "Type": "Label", "Id": "Cochran", "X": 5, "Y": 0 } },
  { "$add": { "Type": "Light", "Block": "cn-mm", "X": 5, "Y": 2 } },
  { "$add": { "Type": "Light", "Block": "cn-ms", "X": 5, "Y": 3 } },
  { "$add": { "Type": "Light", "Block": "cn-e", "X": 6, "Y": 2, "Color": "red", "ShowTrack": false,
              "Interlock": { "Interlock": "cn-e", "SwitchLabels": [ "CE" ], "SignalLabel": "CE" } } },
  { "$add": { "Type": "SwitchLeftBottom", "X": 6, "Y": 2, "Id": "CE" } }
] } }
```

- The siding is drawn below the main (row 3), so the switches branch to the lower right at the west end and come back from the lower left at the east end.
- The two levers sit two columns apart (X 4 and 6), the minimum for one-knob levers.

Example 3: adding a junction to an existing control point

The Robinson Gap mine lead joins the main line just west of Alarka Jct West (`aj-w`). A real railroad would make the new switch part of the existing control point, worked from the same lever. This needs Tetz's SignalsEverywhere Fixes, because `aj-w` shares a module with `aj-e` in the base game; the Fixes mod moves it into its own module, `AJ-W`.

The interlocking: add a switch set, double the routes

```
"AJ-W": {
  "interlocking": {
    "switchSets": { "$replace": [ [ "Nowf" ], [ "Nit5" ] ] },
    "outlets": { "$replace": [
      { "direction": "Left",  "blocks": [ "rg-bk" ],    "nextSignal": "aj-bk-w" },
      { "direction": "Left",  "blocks": [ "rg-mine" ],  "nextSignal": null },
      { "direction": "Right", "blocks": [ "aj-mm" ],    "nextSignal": "aj-em" },
      { "direction": "Right", "blocks": [ "aj-bk-co" ], "nextSignal": null }
    ] },
    "routes": { "$replace": [
      { "switchFilters": [ "Normal",   "Normal" ],   "outletLeft": 0, "outletRight": 2 },
      { "switchFilters": [ "Normal",   "Reversed" ], "outletLeft": 1, "outletRight": 2 },
      { "switchFilters": [ "Reversed", "Normal" ],   "outletLeft": 0, "outletRight": 3 },
      { "switchFilters": [ "Reversed", "Reversed" ], "outletLeft": 1, "outletRight": 3 }
    ] }
  }
}
```

- The stock switch `Nowf` stays set 0, so the stock signals' route numbers keep their meaning for the first switch. The new switch `Nit5` is set 1.
- Two switches, two positions each: four routes. Each route picks a left outlet by the new switch and a right outlet by the old one.
- Everything is `$replace`, never `$find`, so the change counts as a touch (see Known problems).

The signals: new blocks, new mappings

```
"AJ-W": {
  "blocks": {
    "rg": { "spans": [ ... two spans, one per leg of Nit5 ... ] }
  },
  "autoSignals": {
  "aj-wm": { "headConfiguration": "Double",
             "blocks": { "$replace": [ "aj-w", "rg" ] },
             "interlockingRouteMapping": { "$replace": [ 0, 1 ] } },
  "aj-ws": { "headConfiguration": "Double",
             "blocks": { "$replace": [ "aj-w", "rg" ] },
             "interlockingRouteMapping": { "$replace": [ 2, 3 ] } },
  "RG_entry": { "direction": "Right", "headConfiguration": "Double",
                "blocks": [ "rg", "aj-w" ], "interlockingRouteMapping": [ 1, 3 ],
                "location": { "segmentId": "Sjdl", "distance": 20.49, "end": "Start" } }
  }
},
"AJ-BK": {
  "blocks": {
    "rg-bk":   { "spans": [ ... the main west of the junction ... ] },
```

```
    "rg-mine": { "spans": [ ... the mine lead ... ] }
  }
}
```

- The junction's track is a second OS block, `rg`. Every signal at the control point now protects both OS blocks, so a train on either switch holds all of them.
- The new outlet blocks `rg-bk` and `rg-mine` are defined in the plain module `AJ-BK`, not in `AJ-W`. In `AJ-W` they would be OS blocks: a train approaching on the main or coming down the mine lead would cancel the route, and the dispatcher couldn't code one while it waited (see The interlocking's own blocks). An earlier version of this example made that mistake.
- The westbound signals `aj-wm` and `aj-ws` become double: top head for the main to Bryson, second head for the mine.
- `RG_entry` is the new signal on the mine lead, for trains coming out onto the main.
- The westbound home signal `aj-we` stands west of the new switch, so it's moved there and its predicate heads are rewritten to include `Nit5`.
- The old block between Alarka Jct and Bryson is shortened and the new block `rg-bk` takes over its east end. The intermediate on that stretch (`AJ-BK`) is patched to list `rg-bk` and to name the moved `aj-we` as its right-hand end signal, and the intermediate signal facing east (`aj-bk-e`) now protects `rg-bk`.

The panel: two knobs on one lever

```
{ "panel": { "Mainline": [
  { "$find": [ { "path": "Type", "value": "Light" },
               { "path": "X", "value": 39 },
               { "path": "Y", "value": 2 } ],
    "$replace": { "Type": "Light", "Block": "aj-w", "X": 39, "Y": 2,
                  "Color": "red", "ShowTrack": false } },
  { "$add": { "Type": "Light", "Block": "rg", "X": 37.6, "Y": 2, "Color": "red", "ShowTrack": false,
              "Interlock": { "Interlock": "aj-w",
                             "VanillaSwitchKnobIds": [ "35", "aj-w-1" ], "VanillaDirKnobId": "36",
                             "SwitchLabels": [ "RG", "35" ], "SignalLabel": "36",
                             "KnobOrder": [ 1, 0 ] } } },
  { "$add": { "Type": "SwitchLeftBottom", "X": 37.6, "Y": 2, "Id": "RG" } }
] } }
```

- The stock `aj-w` light is replaced by a copy without its `Interlock`, which takes the old lever away from it.
- The lever moves onto the new junction's light. A two-knob lever needs more room, and this puts it between existing columns using fractional `X`.
- `VanillaSwitchKnobIds` keeps the stock knob's id (`35`) so saved switch state still lines up, and gives the new knob its own id.
- `KnobOrder` draws the new switch's knob first, so the knobs read left to right like the track.
- `$find` matches on `Type`, `X` and `Y`, which every element has.

Quick reference

`signals` file

```
{ "<feature>": { "<module>": {
    "blocks":           { "<id>": Block | null },
    "autoSignals":      { "<id>": AutoSignal | null },
    "predicateSignals": { "<id>": PredicateSignal | null },
    "interlocking":     Interlocking,
    "intermediate":     Intermediate,
    "crossover":        Crossover
} | null } }
```

Location

Field	Type	
`segmentId`	string	Track segment id.
`distance`	number	Meters from `end`.
`end`	`Start` | `End`	End measured from. A signal faces this end.

Block

Field	Type	Default	
`spans`	list of `{ lower: Location, upper: Location }`		Track covered.
`thrownSwitchesSetOccupied`	bool	`true`	A hand-thrown switch inside shows the block occupied.
`createInverse`	bool	`false`	Also create `<id>-inv` with the opposite traffic direction.

Every signal

Field	Type	Default	
`direction`	`Left` | `Right`		Direction of travel governed.
`headConfiguration`	`Single` | `Double` | `Triple`		Auto signals: no `Triple`.
`location`	Location		Measure from the end trains approach from.
`leftSide`	bool	`false`	Mast on the left.
`offset`	integer	`3`	Meters from track center (×1.2 on the left).
`modelType`	string	`Vanilla`	Model to copy.

AutoSignal (plus every-signal fields)

Field	Type	
`blocks`	list of block ids	Protected blocks. Occupied or traffic against → Stop.
`interlockingRouteMapping`	list of route indices	One per head, in a control point. `[]` in an intermediate.

PredicateSignal (plus every-signal fields)

Field	Type	
`heads`	list of `{ nextCtcSignal, predicates }`	One per head, top first.
`nextCtcSignal`	signal id | null	Decides green or yellow.
`predicates`	list of Predicate	All must be true.

Predicate `type`	Fields
`Switch`	`switchNode`, `switchSetting` (`Normal` | `Reversed`)
`Block`	`blocks`
`InterlockingTrafficDirection`	`interlocking`, `direction` (`None` | `Left` | `Right`)
`InterlockingTrafficDirectionIsNot`	`interlocking`, `direction`, optional `switchNode` + `switchSetting`
`AlwaysFalse`	none

In a crossover module, a predicate can add `crossoverGroup`.

Interlocking

Field	Type	
`id`	string	Unique railroad-wide.
`displayName`	string	Shown to the player.
`switchSets`	list of lists of node ids	One knob per set.
`outlets`	list of `{ direction, blocks, nextSignal }`	Ways out; direction is of travel leaving.
`routes`	list of `{ switchFilters, outletLeft, outletRight }`	`switchFilters`: `Normal` | `Reversed` | `None`, one per switch set.

Intermediate

Field	Type	
`id`	string	Use the module key.
`blocks`	list of block ids	Left to right.
`signals`	list of signal ids	Left to right, both directions.
`signalLeft`, `signalRight`	signal id	Control point signal past each end.

Crossover

Interlocking fields, plus `routes[].usedBlocks` (list of block ids) and `signalGroups`: list of `{ groupId, signals, allowedDirection, allowedRoutes }`. A `groupId` must be unique across the whole map (its state is saved as `crossover:<groupId>:direction`).

Aspects

Top	Second	Third	Aspect
green			Clear
yellow			Approach
red	green		Diverging Clear
red	yellow		Diverging Approach
red	red	not red	Restricting
red	red	red	Stop

A head that may proceed: green if its next signal isn't at Stop, yellow if it is or there is none.

`ctcPanel` file

```
{ "panel": { "<section>": [ Element, ... ] } }
```

Element field	
`Type`	`Label`, `Light`, `Track`, `TrackRise`, `TrackFall`, `SwitchLeftTop`, `SwitchRightTop`, `SwitchLeftBottom`, `SwitchRightBottom`
`X`	Column order (squashed to 1, 2, 3, ...).
`Y`	Row.
`Block`	Light: block shown. Label: text.
`Id`	Label text; switch highlight id.
`Color`	`white` | `red`
`ShowTrack`	Light: draw track behind. Default `true`.
`SwitchLabel`, `LabelOffsetX`, `LabelOffsetY`	Blue label on a switch piece.
`Interlock`	`{ Interlock, SwitchLabels, SignalLabel, KnobOrder, VanillaSwitchKnobIds, VanillaDirKnobId }`
`Crossover`	`{ Crossover, SwitchKnobOrder, SignalKnobOrder, SwitchLabels, SignalLabels }`

Patch instructions

On	Instruction	
object	`$replace: value`	Replace it.
object	`$moveTo: "['a']['b']"`	Move into an existing object.
property	`$remove: true`	Remove (fails on object values in 1.4).
array item	`$add: value`	Append one.
array item	`$append: [values]`	Append several.
array item	`$find: [conditions]`	Merge into first match.
array item	`$index: n`	Merge into item n.
array item	`$selectAll: true`	With `$find`: every match.
array item	`$replace: value`	With `$find`/`$index`: replace the match.
array item	`$remove: true`	With `$find`/`$index`: remove the match.
array item	`$clone: true`	With `$find`/`$index`: copy, merge, append.
array item	`$optional: true`	With `$find`/`$index`: no match is fine.

Condition: `{ "path": "...", "comparison": "Equals" | "NotEquals" | "StartsWith" | "EndsWith" | "Contains", "value": ... }`. Text compares ignore case. Every item must have the path.

Rules of thumb

- Touch a module's interlocking/intermediate/crossover whenever you patch the module.
- After `$find`/`$index`/`$add` in an existing array, add a harmless `$replace` on the same component.
- When you change a signal, touch every outlet and predicate head that names it.
- Don't `null` predicate signals; move or rewrite them.
- Signals naming another control point go in a module after it.
- Keep `$find` on fields every item has.
- Check the game, not just `signal-patched.json`. Under RailForge, read `## Signal authoring`.